

SUMMER RESEARCH INTERNSHIP
FINAL REPORT

**Adversarial Robustness and
Cross-Victim Attack Transfer**

DeepPoly Certification, Zero Knowledge Exploration,
and Reinforcement Learning

Mentor

Prof. Manik Lal Das

Students

Jeetraj	Student ID: 202303017
Prashit	Student ID: 202303049

DA-IICT, Gandhinagar

May to July 2026

Acknowledgement

We sincerely thank Prof. Manik Lal Das for mentoring us during this Summer Research Internship. His guidance helped us move from an educational implementation of neural network verification to a broader study of privacy and adversarial attack transfer.

The internship gave us room to build, test, make mistakes, and improve our experimental method. We learned that a research result is useful even when the proposed method does not win, as long as the experiment is fair, reproducible, and reported without hiding negative evidence.

We also thank DA-IICT for providing the academic setting for this work.

Abstract

Small changes to an input can cause a neural network to change its prediction. This internship studied that problem from three connected viewpoints. We began with DeepPoly, a method that computes sound lower and upper bounds for neural network outputs. We implemented educational MNIST experiments and explored how certification can be included in training.

We then studied zero knowledge machine learning. The goal was to understand whether a robustness certificate could be checked while keeping a model or test data private. This part remained a research proposal. We did not implement a zero knowledge circuit or generate a proof.

The main experimental phase studied attack transfer with reinforcement learning. A recurrent PPO policy was trained on two source model families, frozen, and evaluated on a held-out victim family. The final CIFAR-10 study used three victim families, three seeds, a 25-query budget, and an L_∞ limit of 8/255. All victim-quality gates passed. However, frozen PPO reached only 1.01% to 1.65% mean attack success across target families and did not beat random search. Its action entropy stayed close to random. A score-guided greedy baseline achieved 13.55% to 27.20%, although its proposal step was not identical to the PPO step.

The final outcome is a working research harness, a carefully audited negative result, and a clear set of next experiments. The study does not show that reinforcement learning can never transfer attacks. It shows that this PPO state, action, and reward design did not learn a useful transferable advantage under the tested conditions.

Contents

Acknowledgement	i
Abstract	ii
1 Introduction	1
2 DeepPoly and Certified Robustness	2
3 Zero Knowledge Exploration	5
4 Reinforcement Learning for Attack Transfer	7
5 Experimental Method	11
6 Results	13
7 Discussion and Next Work	17
8 Conclusion	19

Chapter 1

Introduction

1.1 The question that connected our work

Neural networks can be accurate on normal inputs and still behave unexpectedly after a small change. A few pixels may be changed, a digit may be rotated, or a sound may be slightly perturbed. These changes can be hard for a person to notice but large enough to affect a model.

Our internship followed one central question:

How can we understand, verify, and test a neural network when its input changes slightly?

DeepPoly studies this question from the defender’s side. Zero knowledge proofs add a privacy question. Reinforcement learning studies it from the attacker’s side.

The work did not start as one large system. It grew in stages. Each stage answered one question and exposed the next one.

1.2 Main contributions

The internship produced the following concrete outputs:

- DeepPoly bound propagation and certification-aware training on MNIST;
- PGD, rotation, and speech attack explorations;
- a research map connecting neural verification with zero knowledge machine learning;
- a source-to-target attack framework with frozen and target-adaptation modes;
- a recurrent PPO and GroupDRO training pipeline for cross-victim transfer;
- random, score-bandit, and score-greedy controls under a shared query budget;
- a three-family, three-seed CIFAR-10 study run on an Apple M4;
- audit checks, figures, and reproducible result summaries.

Chapter 2

DeepPoly and Certified Robustness

2.1 Why normal accuracy is not enough

Normal test accuracy measures predictions on a fixed dataset. It does not prove what happens to nearby inputs. Adversarial testing can find a failure, but a failed search does not prove that no failure exists.

Formal certification asks a stronger question. For an input x and radius ε , define

$$\mathcal{B}_\varepsilon(x) = \{x' \in [0, 1]^d : \|x' - x\|_\infty \leq \varepsilon\}. \quad (2.1)$$

The goal is to prove that every x' in this set has the same class as x . This is harder than checking one adversarial example because the set contains infinitely many possible inputs.

2.2 The DeepPoly idea

DeepPoly propagates symbolic lower and upper bounds through the network [4]. Affine layers can be handled with linear expressions. ReLU layers require a relaxation because the function changes form at zero.

For a neuron with pre-activation bounds l and u , there are three cases:

1. If $l \geq 0$, the ReLU is always active and its output equals the input.
2. If $u \leq 0$, the ReLU is always inactive and its output is zero.
3. If $l < 0 < u$, the ReLU is uncertain and is enclosed by sound linear bounds.

For an uncertain ReLU, a standard upper relaxation is

$$z \leq \frac{u}{u-l}(x-l). \quad (2.2)$$

The abstraction may include values that no real input produces, so it can be conservative. However, it must not exclude any valid input in the perturbation set. That soundness is what makes a successful certificate meaningful.

2.3 Certification rule

Suppose class y is the predicted class. Let ℓ_y be a lower bound for its output and u_j be an upper bound for every competing class. The sample is certified when

$$\ell_y > \max_{j \neq y} u_j. \quad (2.3)$$

Equation 2.3 means that class y remains above every alternative for the complete abstract region.

2.4 MNIST certification result

The saved result compares a cross-entropy baseline with a model trained using a DeepPoly-based robustness term. The run evaluated 100 correctly classified MNIST samples.

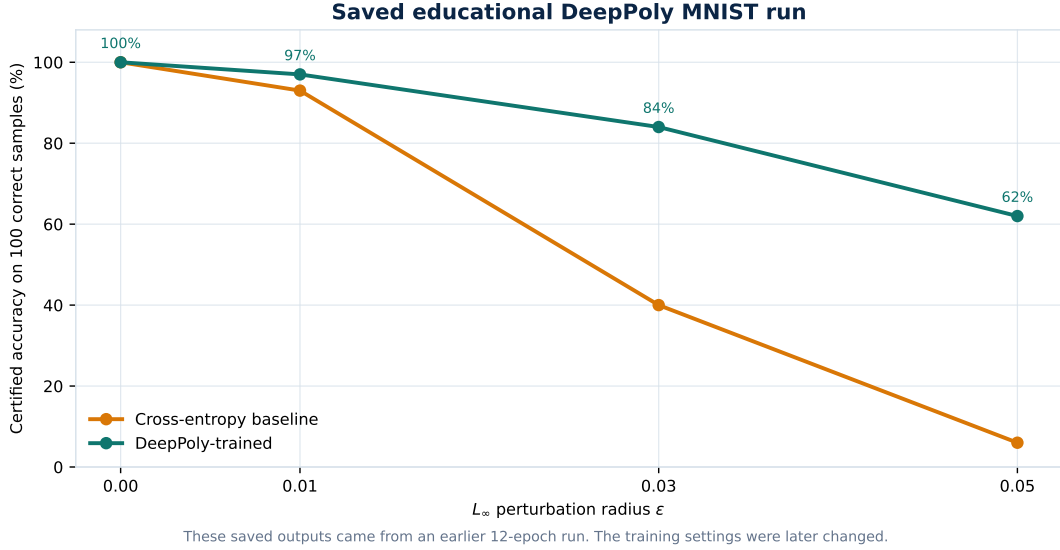


Figure 2.1. Certified accuracy from the saved educational MNIST result.

At $\epsilon = 0.05$, the saved output reported 6% certified accuracy for the baseline and 62% for the DeepPoly-trained model. The average certified margin changed from -5.8963 to 0.7822 . Clean test accuracy was similar: 93.64% for the baseline and 94.12% for the robust model.

Scope of this result. The plot comes from a saved 12-epoch educational run. The training settings were later revised, but a new result was not generated after that change. We therefore treat this as an early experiment rather than a tuned benchmark.

2.5 Certification-aware training

The training experiment combined normal classification loss with a bound-based penalty:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{DP}}. \quad (2.4)$$

Here, $\mathcal{L}_{\text{DP}}$ penalizes cases where a competing upper bound approaches or exceeds the true-class lower bound. The experiment also used an epsilon curriculum, where the training radius increases gradually.

This stage taught us an important distinction. Adversarial training searches for difficult examples and trains on them [3]. Certified training tries to improve a mathematical lower bound on robustness. They are related, but they do not make the same guarantee.

Chapter 3

Zero Knowledge Exploration

3.1 Why privacy entered the project

Formal verification usually assumes that the verifier can access the network and the values used in the calculation. In a real deployment, the model owner may not want to reveal the weights. A data owner may also want to keep evaluation samples private.

Zero knowledge proofs study a different guarantee. A prover can convince a verifier that a computation was carried out correctly without revealing the private witness used in that computation.

Table 3.1. DeepPoly and zero knowledge answer different questions.

	DeepPoly	Zero knowledge proof
Question	Does the network keep its decision for every input in a defined perturbation set?	Was a stated computation checked correctly without revealing the private witness?
Typical private item	Privacy is not the main purpose.	Model weights, data, intermediate values, or other witness values.
Main guarantee	Sound robustness certificate for a defined threat model.	Sound and private verification of a stated relation.

3.2 Proposed certificate-checking workflow

Our research notes proposed separating expensive bound generation from proof checking:

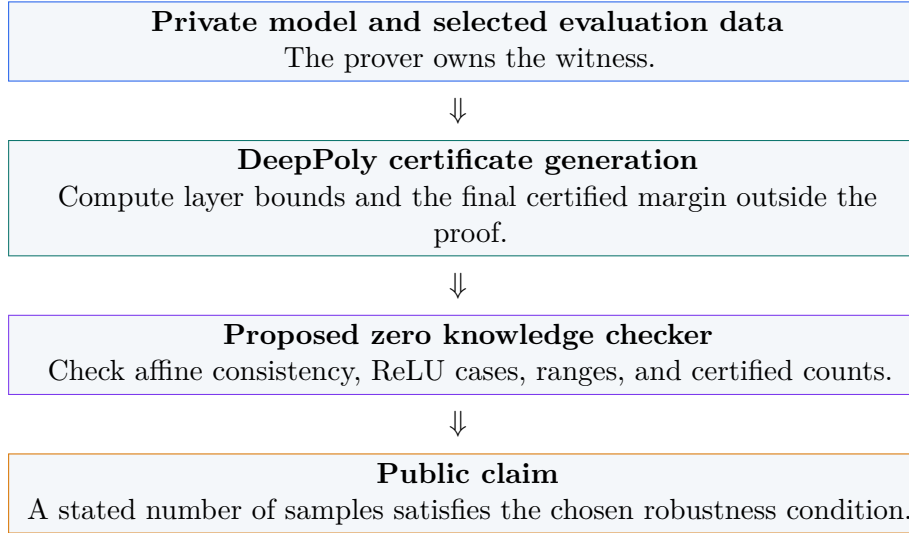


Figure 3.1. Proposed private verification direction. This workflow was designed but not implemented.

The checker would need constraints for affine layers, ReLU phase cases, fixed-point ranges, and the final class separation in Equation 2.3. Recursive proof systems such as Nova [10] and compact range proofs such as Bulletproofs [11] were studied as possible supporting ideas.

Work such as zkCNN and ZEN shows that neural inference can be represented in zero knowledge proof systems [8, 9]. Adding a complete robustness certificate is harder because bound propagation creates many intermediate values and comparison constraints.

Chapter 4

Reinforcement Learning for Attack Transfer

4.1 From verification to attack behavior

DeepPoly asks whether a model is safe throughout a region. Our final experimental direction asked a complementary question: can an attacker learn behavior on some victims and reuse it against a different victim?

Transferability is useful in a realistic black-box setting. The target model may be expensive to query and its parameters may be hidden. An attack policy can train on available source models, then use only a small number of score queries on the target.

4.2 Two modes of transfer

The project supports two related modes. They should not be mixed when interpreting results.

Table 4.1. Frozen transfer and target adaptation.

	Frozen transfer	Target adaptation
Starting point	Policy trained on source victims.	Clone of a policy trained on source victims.
Target-time learning	No parameter update. Recurrent state may change within an episode.	The clone may continue learning on a separate target adaptation subset.
Main question	Did the source policy learn behavior that transfers directly?	Does source training provide a useful starting point for continued learning?
Use in final study	Yes. This is the evaluated scientific protocol.	Implemented as framework support and smoke-tested, but not used for the final headline claim.

For frozen transfer, the policy digest is recorded before and after target evaluation. Matching SHA-256 digests give direct evidence that policy parameters did not change.

4.3 Victim families

The study used three custom CIFAR-10 classifiers. Each victim was trained on 10,000 images for 12 epochs.

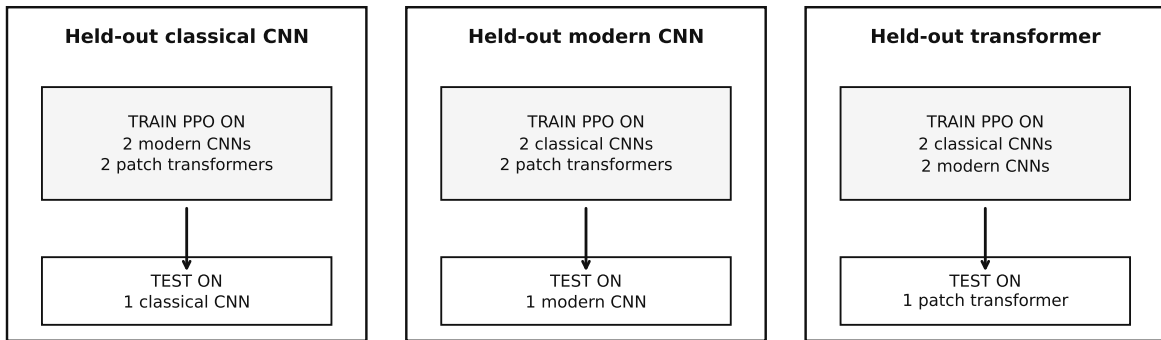
Table 4.2. Victim models used in the study.

Family	Architecture	Parameters
Classical CNN	Residual CNN	2,777,674
Modern CNN	Depthwise CNN	1,223,738
Transformer	Patch transformer	347,722

For each run, PPO trained against four source victims: two instances from each of two families. One model from the third family was kept for frozen evaluation. The three held-out settings were repeated with seeds 17, 29, and 41.

Each run used 1,000 policy-training images and 1,000 source-validation images. PPO scheduled 600 attack episodes, updated after every block of 25 episodes, and used four optimization passes per update. Final evaluation used 300 separate images.

Models used to train and test the frozen PPO attacker



Each victim: 10,000 training images and 12 epochs | PPO: 1,000 policy images and 600 scheduled episodes

Final evaluation: 300 separate images and 25 calls per image

Seeds 17, 29, and 41 produce 9 separately trained PPO policies and 9 frozen evaluations

Figure 4.1. The exact source and target model arrangement used in the nine runs.

4.4 RL agent structure

The recurrent PPO policy [5] does not receive the raw image. It receives eight response features: correct-class rank, entropy, confidence change, confidence margin, remaining budget, previous action, previous reward, and attack progress.

The encoder maps these 8 values to 96 learned features. A 96-unit GRU carries memory across attack steps. The shared representation then splits into an actor and a critic. The actor produces 96 action logits, while the critic produces one value estimate.

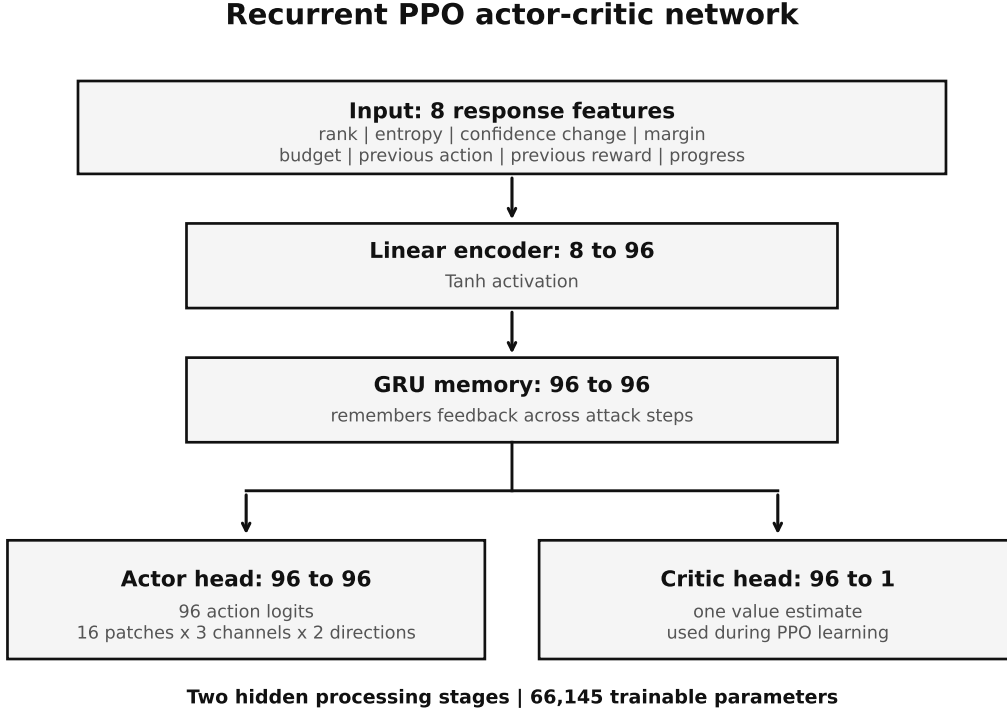


Figure 4.2. The recurrent PPO actor-critic network.

The complete policy has 66,145 trainable parameters. Its two hidden processing stages are the 96-unit encoder and the 96-unit GRU memory layer.

The image is divided into 16 patches. An action selects:

- one of 16 patches;
- one of three color channels;
- a positive or negative direction.

The resulting discrete action space has

$$16 \times 3 \times 2 = 96 \quad (4.1)$$

actions. A PPO action proposes a step of $2/255$. Every update is projected back into the allowed L_∞ region and clipped to valid image values:

$$x_{t+1} = \text{clip}_{[0,1]} \left(\text{clip}_{[x-\varepsilon, x+\varepsilon]} (x_t + \delta(a_t)) \right). \quad (4.2)$$

4.5 Reward and source-family balance

Sparse success rewards gave too little learning signal in early experiments. The final version used the true-class margin

$$m_t = p_y(x_t) - \max_{j \neq y} p_j(x_t), \quad (4.3)$$

and the dense reward

$$r_t = 5(m_{t-1} - m_t) + 2\mathbb{I}[f(x_t) \neq y] - 0.01. \quad (4.4)$$

The first term rewards margin reduction, the second rewards a successful misclassification, and the last term gives a small cost for using a step.

GroupDRO weighting was used to keep training from focusing only on the easiest source family [6]. Family losses and source-model calls were recorded for audit.

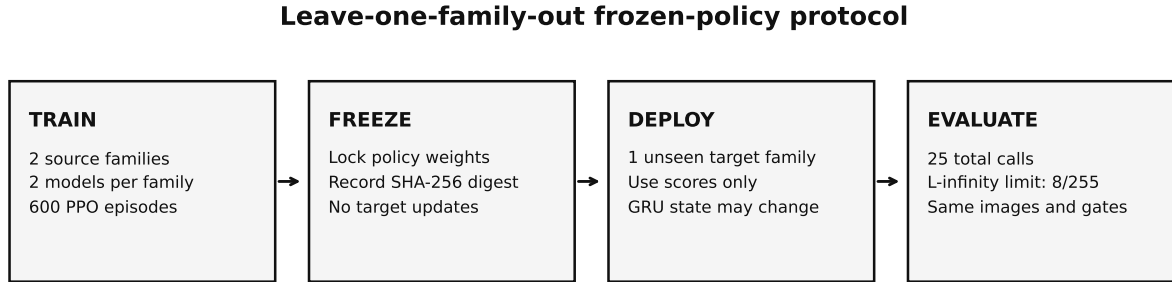
Chapter 5

Experimental Method

5.1 Threat model and query accounting

The target evaluation used:

- CIFAR-10 images [12];
- an L_∞ limit of 8/255;
- 25 total target score calls, including initialization;
- only images correctly classified by the clean target model;
- the same eligible images for all compared methods;
- no target-family policy training or tuning in the frozen protocol.



Repeat for classical CNN, modern CNN, and transformer targets with seeds 17, 29, and 41.

Figure 5.1. Training, freezing, deployment, and evaluation under the final protocol.

Every target call counts. Failed attempts are not removed. Lower-query measurements come from the same trajectory, so a method does not receive extra attempts at each query checkpoint.

5.2 Controls

Four methods were compared:

1. **Stochastic PPO:** sample actions from the frozen learned policy.

2. **Random:** sample uniformly from the same action catalog.
3. **Score bandit:** use observed score changes to favor useful actions.
4. **Score greedy:** test score-guided patch proposals and keep improvements.

The score-greedy control shares the image set, final L_∞ limit, action catalog, and overall target-query budget. Its direct proposal magnitude is 8/255, while PPO uses 2/255 steps. It is therefore a strong reference for score-guided patch search, but not a perfectly matched comparison of identical update operators.

5.3 Metrics

The main metric is attack success rate:

$$\text{ASR}(q) = \frac{\text{successful attacks by query } q}{\text{clean-correct eligible target images}}. \quad (5.1)$$

We also measured the area under the ASR-by-query curve, confidence-margin reduction, action entropy, runtime, victim accuracy, and policy-digest stability.

The promotion rule was deliberately strict. It checked practical and statistical improvement over controls, acceptable action entropy for every seed, victim-quality gates, shared samples, exact budgets, and internal consistency. A missing or failed check caused the result to fail closed.

5.4 Hardware and scale

The full experiment ran locally on an Apple M4 with MPS acceleration. It completed in 6,864.9 seconds, or about 114.4 minutes.

Table 5.1. Scale of the final CIFAR-10 study.

Item	Completed value
Held-out target families	3
Fresh seeds	17, 29, 41
Family and seed folds	9
Scheduled training episodes	5,400
Trainable sequences	3,640
Source-model calls	91,800
Clean-correct target image and run cases per method	1,859
Total target calls per image and method	25
Wall-clock time	114.4 minutes

Chapter 6

Results

6.1 Victim-quality checks

A transfer study is not useful if the victim models are too weak to classify clean data. Each family therefore had a minimum validation-accuracy gate. All nine folds passed.

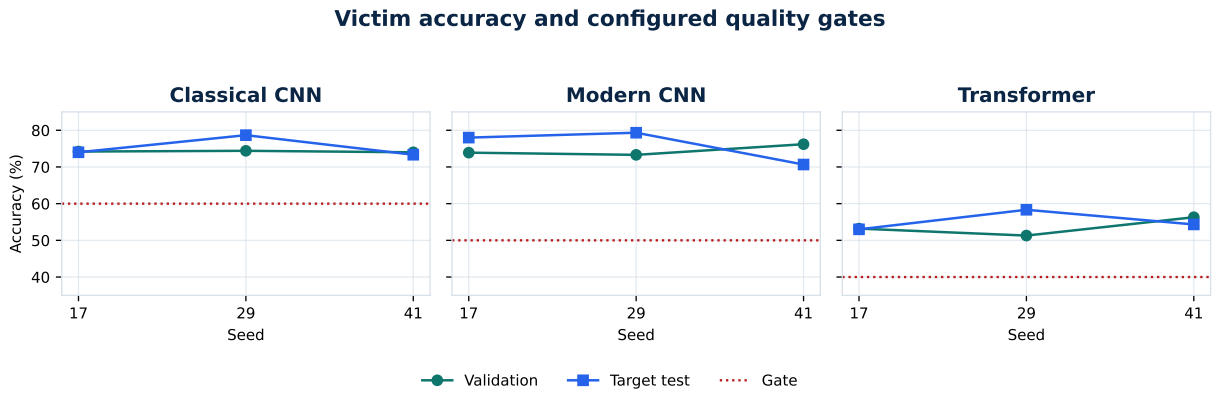


Figure 6.1. Validation accuracy, target-test accuracy, and configured quality gates.

Table 6.1. Victim accuracy across the three study seeds.

Family	Validation range	Gate	Target-test range
Classical CNN	74.0% to 74.4%	60%	73.3% to 78.7%
Modern CNN	73.3% to 76.2%	50%	70.7% to 79.3%
Transformer	51.3% to 56.3%	40%	53.0% to 58.3%

6.2 Final attack success

Figure 6.2 gives the across-seed mean ASR at 25 total target calls.

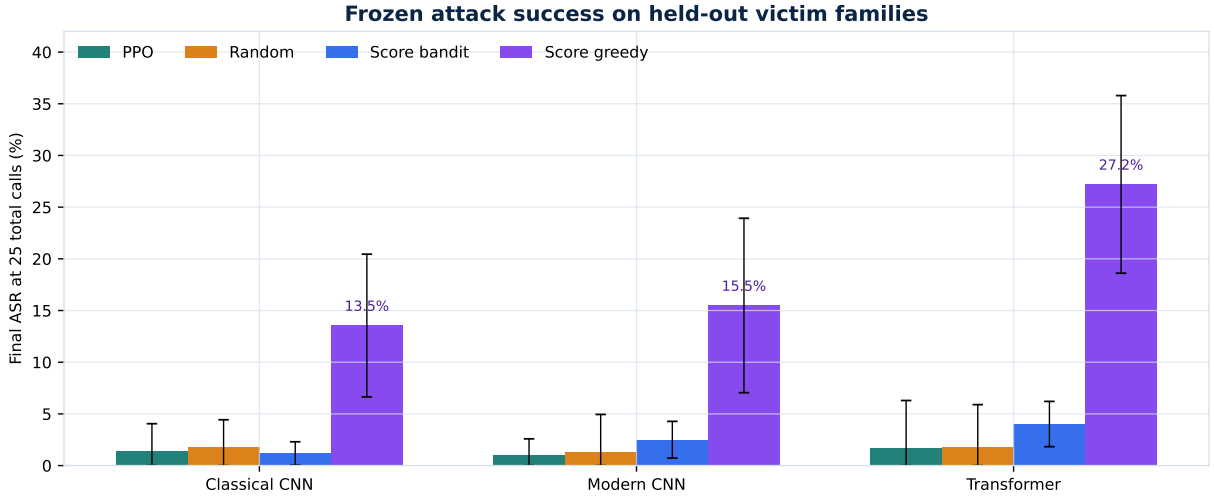


Figure 6.2. Mean final attack success rate across seeds 17, 29, and 41.

Table 6.2. Mean final ASR on each held-out target family.

Held-out target	PPO	Random	Score bandit	Score greedy
Classical CNN	1.45%	1.74%	1.17%	13.55%
Modern CNN	1.01%	1.29%	2.50%	15.49%
Transformer	1.65%	1.84%	4.02%	27.20%

PPO did not beat random on any family mean. It was slightly above the score-bandit mean on the classical target, but the full paired promotion rule still failed. Score greedy was much stronger on all three target families.

6.3 Success across the query sequence

Final ASR gives one endpoint. Figure 6.3 shows when successes appeared during the 25-call budget.

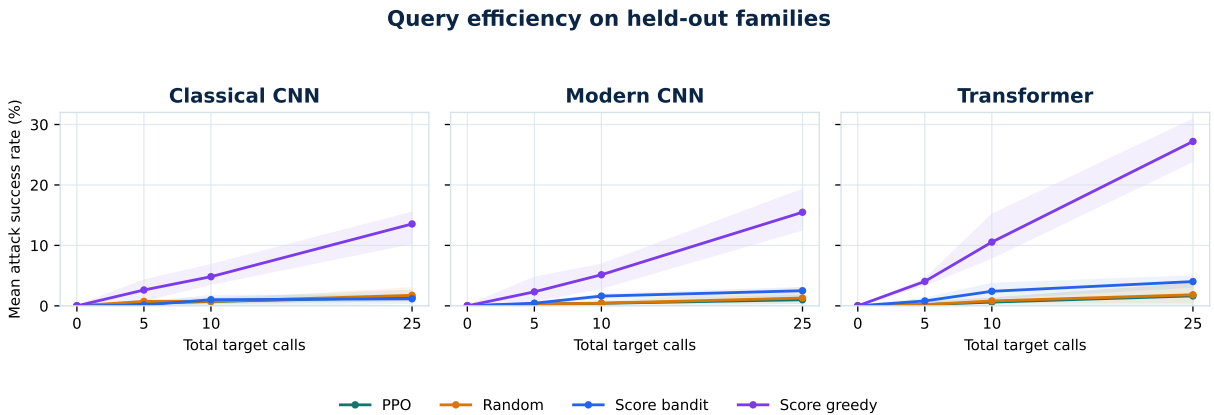


Figure 6.3. Across-seed ASR as the target-query budget is used.

The ASR-by-query area values were:

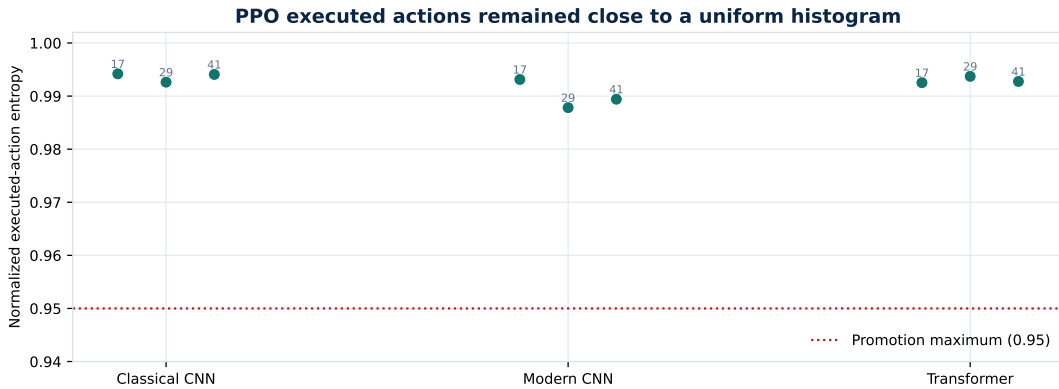
Table 6.3. Area under the ASR-by-query curve. Higher is better.

Held-out target	PPO	Random	Score bandit	Score greedy
Classical CNN	0.81	0.96	0.79	6.52
Modern CNN	0.53	0.62	1.48	7.17
Transformer	0.78	0.91	2.33	13.19

The endpoint and query-curve results tell the same story. PPO did not gain an early or sustained advantage over random actions. Score-guided selection used the limited calls more effectively.

6.4 What the action entropy revealed

Normalized executed-action entropy measures how widely the policy spreads its selected actions. A value close to one indicates behavior close to a uniform action distribution.

**Figure 6.4.** PPO executed-action entropy across target families. The promotion maximum was 0.95.

PPO entropy stayed between 0.988 and 0.994. This exceeded the promotion maximum of 0.95 for every family. The learned policy had not developed a strong action preference, which agrees with its near-random ASR.

6.5 Pilot and full-study comparison

The first M4 pilot trained on a classical CNN and a modern CNN, then attacked a held-out patch transformer. It finished in 6 minutes 12 seconds. On 99 clean-correct images, stochastic PPO achieved 5.1% ASR and random achieved 7.1%.

The pilot was useful because it verified the frozen-policy digest and the end-to-end MPS workflow. It also warned us not to claim an RL advantage from a single small run.

The full study improved the reward, victim fitting, source populations, controls, seeds, and audit rules. The stronger experiment reached the same broad conclusion: PPO did not establish a transfer advantage.

6.6 Interpretation

Main empirical conclusion. Under this CIFAR-10 setup, the recurrent GroupDRO PPO policy did not learn a useful frozen cross-victim advantage. All victim-quality gates passed, but the promotion gate failed for every held-out family.

This is a result about the tested design, not a universal result about reinforcement learning or attack transfer. The state was compact, the action catalog was coarse, the query budget was small, and only three custom victim families were used.

The score-greedy result is still informative. It shows that score-guided patch search can succeed in the same overall threat setting. It also suggests that the PPO policy did not extract enough value from score changes. Because score greedy used a larger direct proposal step, matched step-size ablations are needed before assigning the full gap to its search rule.

6.7 Why the PPO success rate remained low

The measurements point to a learning problem rather than a failed evaluation. All victim-quality gates passed, the policy stayed frozen during target testing, and score greedy found successful attacks on the same eligible images. The following evidence suggests likely causes, but only matched ablations can establish their separate effects.

Table 6.4. Measured evidence and likely effects on PPO learning.

Measured evidence	What it suggests
Normalized PPO action entropy stayed between 0.988 and 0.994.	The actor remained close to a uniform distribution over 96 actions. It did not learn a reliable preference for useful patches, channels, or directions.
Only 64 of 3,640 trainable source episodes ended in success, a rate of 1.76%.	The terminal success bonus was rare. Most updates depended on small confidence-margin changes, which can be noisy and different across victims.
Only 3,640 of 5,400 scheduled episodes produced PPO sequences, or 67.4%.	The other episodes were skipped because the selected source victim was already wrong on the clean image. The effective amount of policy training was therefore smaller than the schedule suggests.
The observation contained eight global score features.	It did not contain image content, a perturbation map, patch visit counts, or a full record of earlier patch effects. Different attack situations could therefore look similar to the policy. The GRU can remember received information, but it cannot recover information that was never observed.
PPO used a fixed 2/255 step and had 24 attack calls after initialization.	Reaching the 8/255 boundary in one direction required repeated compatible choices. A nearly uniform policy could spend much of the small query budget on weak or conflicting steps.
PPO kept every proposed move, while score greedy kept only margin-reducing moves and proposed 8/255 changes.	PPO could undo earlier progress. The stronger greedy result combines a score-guided search rule, rollback behavior, and a larger proposal, so the present experiment cannot assign the full gap to only one of these factors.
Each run used four source victims from two families and one unseen target victim.	This is enough to test the protocol, but it offers limited architectural diversity. Patch preferences learned on the source victims may not remain useful on the held-out family.

Chapter 7

Discussion and Next Work

7.1 Main lessons

- A successful attack shows that one failure exists. A sound certificate can prove robustness for every input inside a defined region.
- Frozen transfer and continued target learning are different experiments. The final result measures frozen transfer only.
- Controls are essential. PPO reached 5.1% in the pilot, but random search reached 7.1%.
- Near-random action entropy showed that the policy had not learned a selective attack strategy.

7.2 Limitations

- The DeepPoly result used 100 correctly classified MNIST samples from an early training configuration.
- The zero knowledge part remained a proposal and was not implemented.
- The RL study used CIFAR-10, three custom victim families, and three seeds.
- Each fold used one held-out target instance.
- PPO and score greedy used different proposal magnitudes.
- The result should not be generalized to larger pretrained models.

7.3 How to improve RL attack success

The next study should improve the learning signal, state representation, action design, and source diversity in a controlled order.

1. **Match the proposal operator.** Give PPO, random, bandit, and greedy methods the same step sizes and the same accept-or-reject rule. Test 2/255, 4/255, and 8/255 as separate ablations. This will show whether the current gap comes mainly from action selection or from the stronger greedy proposal.
2. **Keep the best candidate found so far.** After a query, retain a proposal only when it reduces the correct-class margin. A rejected proposal still consumes its query, but it does not damage the next state. The policy can also receive an explicit accepted-or-rejected feature.

3. **Start from successful demonstrations.** Collect score-greedy trajectories on source victims and use behavior cloning to train the actor before PPO updates begin. PPO can then improve a selective starting policy instead of beginning from almost uniform actions.
4. **Give the policy a richer observation.** Add patch visit counts, the signed score change caused by each recent action, current perturbation level per patch, recent accepted actions, and a short window of margin changes. A compact image or patch embedding can also be tested because it does not require access to target-model parameters.
5. **Use hierarchical and multi-scale actions.** Let the policy choose a region, channel, direction, and step size in stages. Include coarse and fine patch grids. This can support large early moves, smaller refinements near the decision boundary, and clearer learning than one flat 96-action decision.
6. **Increase the amount and quality of source training.** Build each episode from an image that the assigned source victim classifies correctly, so scheduled episodes are not lost. Begin with low-margin images, then move toward harder samples. Add more independently trained source instances and more architecture families after the policy beats random on source validation.
7. **Tune exploration and reduce update variance.** Start with useful exploration and gradually reduce the entropy coefficient. Compare reward normalization per victim, generalized advantage estimation, learning-rate sweeps, and a larger success bonus. Monitor entropy, source ASR, validation ASR, critic error, and gradient norms so a uniform or collapsed policy is detected early.
8. **Compare learning algorithms under the same operator.** If PPO remains uniform, compare it with a recurrent DQN and a score-aware contextual bandit. This checks whether the limitation comes from the PPO update or from the state and action design shared by all methods.
9. **Evaluate target adaptation separately.** Clone the source policy, fine-tune the clone on a disjoint target-adaptation subset, and test it on untouched target images. Keep frozen transfer as a separate result. This measures whether source training provides a useful starting point even when direct frozen transfer is weak.

7.3.1 Recommended evaluation order

1. Run matched step-size and rollback ablations using the current trained policies and controls.
2. Train a behavior-cloned policy from successful source-side greedy trajectories.
3. Add richer observations and hierarchical actions one change at a time.
4. Tune PPO on source validation until it beats matched random search and its entropy is neither uniform nor collapsed.
5. Repeat frozen leave-one-family-out evaluation with at least three seeds, then run the separate target-adaptation protocol.

The primary result should remain ASR at 25 total calls. Results at 50 or 100 calls can be added as secondary budget ablations to show whether the method needs more feedback, but they should not replace the original query-matched comparison.

Chapter 8

Conclusion

This internship began with a direct implementation question: how can DeepPoly certify an MNIST model around a small input region? From there, the work expanded into certification-aware training, adversarial examples, private verification ideas, and finally cross-victim RL attack transfer.

The DeepPoly stage gave us a working understanding of sound bounds and certified margins. The zero knowledge stage showed how robustness and privacy could meet, while also making clear how much engineering remains before such a system can be claimed as implemented.

The RL stage became the largest experimental contribution. We built a frozen-policy protocol, a target-adaptation path, recurrent PPO training, multiple controls, audit checks, M4 execution, and a nine-fold result. The final outcome did not support the original hope that PPO would learn a transferable advantage. PPO stayed near random, while score-greedy search was much stronger.

That negative result is a useful conclusion to the internship. It narrows the next research question from “Does RL transfer?” to “What state, action, and supervision would make a transfer policy selective and useful?” The evidence supports a concrete next path: match the proposal operators, learn first from successful score-guided trajectories, retain the best candidate, enrich the observation, and only then expand training and target adaptation. The completed work provides the evidence and tools needed to study that question carefully.

Bibliography

- [1] C. Szegedy et al., “Intriguing properties of neural networks,” International Conference on Learning Representations, 2014.
- [2] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” International Conference on Learning Representations, 2015.
- [3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” International Conference on Learning Representations, 2018.
- [4] G. Singh, T. Gehr, M. Püschel, and M. Vechev, “An abstract domain for certifying neural networks,” Proceedings of the ACM on Programming Languages, vol. 3, POPL, 2019.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [6] S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang, “Distributionally robust neural networks for group shifts,” International Conference on Learning Representations, 2020.
- [7] C. Guo, J. R. Gardner, Y. You, A. G. Wilson, and K. Q. Weinberger, “Simple black-box adversarial attacks,” International Conference on Machine Learning, 2019.
- [8] T. Liu, X. Xie, and Y. Zhang, “zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy,” ACM SIGSAC Conference on Computer and Communications Security, 2021.
- [9] B. Feng et al., “ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences,” Cryptology ePrint Archive, Report 2021/087, 2021.
- [10] A. Kothapalli, S. Setty, and I. Tzialla, “Nova: Recursive zero-knowledge arguments from folding schemes,” Annual International Cryptology Conference, 2022.
- [11] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, “Bulletproofs: Short proofs for confidential transactions and more,” IEEE Symposium on Security and Privacy, 2018.
- [12] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Technical report, University of Toronto, 2009.